24. Trees, Heaps, and Priority Queues

24. Trees, Heaps, and Priority Queues

Phase: Phase 4 — Algorithms and data structures
Estimated time: 2 days
Slug: `24-trees-heaps`

Learning objectives

- Implement binary tree traversals
- Use heap.Interface for priority queues
- Know BST search costs
- Apply heaps to scheduling problems

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Binary tree

```
type Node struct {
    Val int
    Left, Right *Node
}
```

Traversals: inorder, preorder, postorder, level-order (BFS with queue).

Heaps / priority queues

`container/heap` needs `Len`, `Less`, `Swap`, `Push`, `Pop`.

Heaps give O(log n) push/pop and O(1) peek — perfect for: - scheduling jobs by priority - Dijkstra's algorithm - top-K queries

BST intuition

Balanced BST: O(log n) search. Unbalanced (sorted inserts): O(n). Production often uses maps (hash) or specialized trees.

Common mistakes (especially from Python instincts)

- Assuming BST is always log n
- Incorrect Less making heap invalid
- Mutating heap slice outside heap functions

Exercises

Work in `exercises/24-trees-heaps/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.

- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/24-trees-heaps/` until you have an honest failing attempt.

Self-check

1. Complexity of heap push?
2. Name three traversals
3. When use priority queue in infra?

Further reading

- https://pkg.go.dev/container/heap